

SEMANA 1

Trabajo práctico 01 - Fundamentos de Node.js

1. Información y entrega

Aspecto	Detalle
Modalidad	Individual
Puntaje	10 puntos
Vencimiento	Lunes siguiente a las 19:59
Entrega	URL del repositorio individual de GitHub
Nombre sugerido	tp-01-fundamentos-node

El repositorio debe poder ejecutarse y comprenderse sin una explicación oral adicional.

2. Consigna

Construir un proyecto que reúna tres evidencias de la primera semana:

- `index.js` : genera la ficha de un videojuego y la guarda en un archivo.
- `orden-event-loop.js` : demuestra el orden de ejecución trabajado en clase.
- `README.md` : explica cómo ejecutar el proyecto y responde cuatro preguntas conceptuales.

El ejemplo debe utilizar datos propios. No se debe copiar la película o el libro de las clases cambiando únicamente algunas palabras.

3. Estructura del proyecto

La entrega final debe tener esta estructura:

```
tp-01-fundamentos-node/  
|-- .gitignore  
|-- index.js  
|-- orden-event-loop.js  
|-- README.md  
|-- salida/  
    |-- ficha-videojuego.txt
```

La carpeta `salida` y el archivo `ficha-videojuego.txt` deben ser creados por `index.js` , no manualmente.

4. Parte 1 - Ficha de un videojuego

Crear `index.js` con los siguientes requisitos.

Datos del runtime

- Recibir el nombre del estudiante mediante `process.argv` .
- Utilizar un nombre alternativo cuando no se proporcione un argumento.
- Mostrar `process.version` y `process.platform` .

El programa debe funcionar de ambas formas:

```
node index.js
node index.js Camila
```

Datos del videojuego

Crear un objeto con estas propiedades:

Propiedad	Tipo	Requisito
<code>titulo</code>	Texto	Título del videojuego
<code>estudio</code>	Texto	Estudio o empresa responsable
<code>anio</code>	Número	Año de publicación
<code>plataformas</code>	Arreglo	Al menos dos elementos
<code>multijugador</code>	Booleano	<code>true</code> o <code>false</code>

Antes de construir la ficha:

- Convertir `plataformas` en un texto mediante `join` .
- Convertir `multijugador` en "Sí" o "No" mediante un operador condicional.
- Reunir los datos en un template literal.

La ficha puede personalizarse, pero debe incluir como mínimo:

```
FICHA DE VIDEOJUEGO
=====
Estudiante: ...
Node.js: ...
Plataforma del sistema: ...

Titulo: ...
Estudio: ...
Año: ...
Plataformas: ...
¿Es multijugador?: ...
```

Creación del archivo

Utilizar:

- `node:path` y `path.join` para construir las rutas.
- `node:fs` y `fs.mkdirSync` con `{ recursive: true }` para crear `salida` .
- `fs.writeFileSync` con codificación `utf8` para generar `salida/ficha-videojuego.txt` .

El programa debe:

1. Mostrar la ficha en la terminal.
2. Guardar la misma información en el archivo.

3. Informar la ruta del archivo generado.
4. Poder ejecutarse nuevamente sin errores.

5. Parte 2 - Orden del event loop

Crear `orden-event-loop.js` con exactamente tres mensajes:

1. Un mensaje que indique el comienzo del programa.
2. Un `setTimeout` con tiempo `0` cuyo callback muestre el tercer mensaje.
3. Un mensaje, ubicado después del temporizador, que indique el final del código principal.

Ejecutar:

```
node orden-event-loop.js
```

La salida debe respetar este orden conceptual:

1. Comienza el programa
2. Termina el código principal
3. Se ejecuta la tarea programada

Los textos pueden cambiar, pero el resultado debe demostrar que el callback no interrumpe el código principal.

6. README y repositorio

El `README.md` debe incluir:

```
# Trabajo práctico 01
## Descripción
## Cómo ejecutar
## Archivo generado
## Conceptos
```

En esas secciones se debe indicar:

- Qué construye el proyecto.
- Cómo ejecutar ambos programas.
- Dónde se genera `salida/ficha-videojuego.txt` .
- Las respuestas, con palabras propias, a estas preguntas:
 1. ¿Qué diferencia existe entre JavaScript, V8 y el runtime de Node.js?
 2. ¿Por qué el callback de `setTimeout(..., 0)` se ejecuta después del código principal?
 3. ¿Cuál es la diferencia general entre I/O bloqueante y no bloqueante?
 4. ¿Qué responsabilidades cumplen `node:path` y `node:fs` en `index.js` ?

Crear también `.gitignore` :

```
node_modules/
.env
```

No se deben instalar paquetes de NPM ni utilizar servidores, bases de datos, promesas o `async/await` . Se permiten los apuntes, los ejercicios de la semana y la documentación oficial enlazada en ellos.

7. Criterios de evaluación

Criterio	Puntaje
Uso de <code>process</code> y argumento opcional	1 punto
Objeto, transformaciones y ficha	2 puntos
Creación del archivo con <code>node:path</code> y <code>node:fs</code>	2 puntos
Demostración del event loop	1 punto
README y explicaciones conceptuales	2 puntos
Organización, funcionamiento y entrega	2 puntos
Total	10 puntos

Para aprobar, ambos programas deben ejecutarse sin errores, la ficha debe generarse automáticamente y el repositorio debe estar accesible para el equipo docente.

8. Comprobación y entrega

- ☐ `index.js` funciona con y sin el nombre del estudiante.
- ☐ Se muestran la versión de Node.js y la plataforma.
- ☐ El objeto y las transformaciones cumplen los requisitos.
- ☐ La carpeta y la ficha se generan automáticamente.
- ☐ La ficha aparece en la terminal y en el archivo.
- ☐ Una segunda ejecución funciona sin errores.
- ☐ `orden-event-loop.js` muestra los tres mensajes en el orden solicitado.
- ☐ El README incluye los comandos y las cuatro respuestas.
- ☐ `.gitignore` excluye `.env` y `node_modules`.
- ☐ El repositorio está actualizado y puede abrirse desde la URL entregada.

Antes del vencimiento, ejecutar ambos programas desde una terminal nueva, revisar el archivo generado, sincronizar el repositorio y enviar su URL. Si el repositorio es privado, se debe otorgar acceso al equipo docente.